CSE232 Prelab 2: Loops and Indexed Addressing

Not graded | Guided exercises with solutions

How to Use This Prelab

This prelab teaches the two ideas you need for Labwork 2: **loops** and **indexed addressing**. Each task shows the same program in C++, gives you a plan, and gives you a code skeleton with blanks (`____`) to fill in. Try each task in the simulator before you look at the solutions at the end.

Key Idea 1: Arrays

An array is a block of data where each data is stored consecutively. If create an array with 5 elements starting from the address $60, we need to put our values at adresses $60, $61, $62, $63 and $64.

Key Idea 2: Indexed Addressing

In indexed addressing the CPU computes the address while the program runs:

address = offset + X

If the offset is the start address of an array and X is the index i, then `LDAA $40,X` means `A = arr[i]`:

X	LDAA $40,X reads	C++
0	$40	`arr[0]`
1	$41	`arr[1]`
2	$42	`arr[2]`

Key Idea 3: A Loop Over an Array

A C++ `for` loop and a 6800 loop are built from the same parts:

Part	C++	6800
Initialize	`i = 0`	`LDX #0` before the loop
Body	the work inside `{ }`	the instructions after the `LOOP` label
Update	`i++`	`INX`
Test	`i < N`	`CPX #N` then `BNE LOOP`

`INX` increases the X by 1, then the **same instructions** reads a different element in every pass of the loop. Because we know the size of the array, X itself tells us when to stop: after `INX`, `CPX #N` checks whether X has reached N. After `CPX`, we can use `BNE` to continue the loop if X did not reach the N.

New instructions

- `LDX #0`: X = 0 (i = 0)
- `LDAA $40,X`: load A the value at address (40 + X)
- `ADDA $40,X`: A = A + element at address (40 + X)
- `INX`: X = X + 1 (i++)
- `CPX #3`: compare X with 3
- `BNE TAG`: If i is not 3, go to the TAG label

Task 1: Copy an Array

Copy 3 elements from `$40–$42` to `$45–$47`.

Memory: `$40` = 15, `$41` = 8, `$42` = 22

The same program in C++

```
for (i = 0; i < 3; i++)
    dst[i] = src[i];
```

Plan

1. Set X = 0.
2. Load `src[i]` from `$40,X`.
3. Store it to `dst[i]` at `$45,X`.
4. Increase X. If X is not 3, go back to step 2.

Fill in the blanks

```
        LDAA #15        ; test input
        STAA $40
        LDAA #8
        STAA $41
        LDAA #22
        STAA $42

        LDX  ____       ; step 1: i = 0
LOOP    LDAA ____       ; step 2: A = src[i]
        STAA ____       ; step 3: dst[i] = A
        INX             ; step 4: i++
        CPX  ____       ;         i == 3 ?
        BNE  LOOP       ;         no: repeat
        .END
```

Check yourself: `$45` = 15, `$46` = 8, `$47` = 22.

Task 2: Double Every Element

Read 4 elements from `$50–$53`, multiply each by 2 and store the results at `$60–$63`.

Memory: `$50` = 4, `$51` = 7, `$52` = 2, `$53` = 10

The same program in C++

```
for (i = 0; i < 4; i++)
    b[i] = a[i] * 2;
```

Plan

1. Set X = 0.
2. Load `a[i]` from `$50,X`.
3. Shift A left with `ASLA` (multiply by 2, as in Lab 1).
4. Store A to `b[i]` at `$60,X`.
5. Increase X. If X is not 4, go back to step 2.

Fill in the blanks

```
        LDAA #4         ; test input
        STAA $50
        LDAA #7
        STAA $51
        LDAA #2
        STAA $52
        LDAA #10
        STAA $53

        LDX  #0         ; step 1: i = 0
LOOP    LDAA ____       ; step 2: A = a[i]
        ____            ; step 3: A = A * 2
        STAA ____       ; step 4: b[i] = A
        INX             ; step 5: i++
        CPX  ____       ;         i == 4 ?
        BNE  LOOP
        .END
```

Check yourself: `$60` = 8, `$61` = 14, `$62` = 4, `$63` = 20.

Task 3: Sum of an Array

Add up the 4 elements stored at `$70`–`$73` and store the sum at `$74`.

Memory: `$70` = 6, `$71` = 3, `$72` = 9, `$73` = 2

The same program in C++

```
sum = 0;
for (i = 0; i < 4; i++)
    sum = sum + arr[i];
```

Plan

1. Set the sum (A) to 0 and X = 0.
2. Add `arr[i]` to A with `ADDA $70,X`.
3. Increase X. If X is not 4, go back to step 2.
4. Store A at `$74`.

Fill in the blanks

```
        LDAA #6         ; test input
        STAA $70
        LDAA #3
        STAA $71
        LDAA #9
        STAA $72
        LDAA #2
        STAA $73

        ____            ; step 1: sum = 0
        LDX  #0         ;         i = 0
LOOP    ADDA ____       ; step 2: sum = sum + arr[i]
        INX             ; step 3: i++
        CPX  ____       ;         i == 4 ?
        BNE  LOOP
        STAA ____       ; step 4: store the sum
        .END
```

Check yourself: $74 = 20 ($14).

Solutions

Read this section only after you have tried the tasks.

Solution for Task 1 (Copy an Array)

```
        LDAA #15
        STAA $40
        LDAA #8
        STAA $41
        LDAA #22
        STAA $42

        LDX  #0         ; i = 0
LOOP    LDAA $40,X      ; A = src[i]
        STAA $45,X      ; dst[i] = A
        INX             ; i++
        CPX  #3         ; i == 3 ?
        BNE  LOOP
        .END
```

Final values: `$45` = 15, `$46` = 8, `$47` = 22

Sample trace

Iteration	X	Load address	A (loaded)	Store address	Memory change
1	0	$40	15	$45	$45 = 15
2	1	$41	8	$46	$46 = 8
3	2	$42	22	$47	$47 = 22

After iteration 3, X becomes 3, `CPX #3` finds them equal and the loop ends.

Solution for Task 2 (Double Every Element)

```
        LDAA #4
        STAA $50
        LDAA #7
        STAA $51
        LDAA #2
        STAA $52
        LDAA #10
        STAA $53

        LDX  #0         ; i = 0
LOOP    LDAA $50,X      ; A = a[i]
        ASLA            ; A = A * 2
        STAA $60,X      ; b[i] = A
        INX             ; i++
        CPX  #4         ; i == 4 ?
        BNE  LOOP
        .END
```

Final values: `$60` = 8, `$61` = 14, `$62` = 4, `$63` = 20

Sample trace

Iteration	X	A (loaded)	A (after ASLA)	Store address	Memory change
1	0	4	8	$60	$60 = 8
2	1	7	14	$61	$61 = 14
3	2	2	4	$62	$62 = 4
4	3	10	20	$63	$63 = 20

Solution for Task 3 (Sum of an Array)

```
        LDAA #6
        STAA $70
        LDAA #3
        STAA $71
        LDAA #9
        STAA $72
        LDAA #2
        STAA $73

        CLRA            ; sum = 0
        LDX  #0         ; i = 0
LOOP    ADDA $70,X      ; sum = sum + arr[i]
        INX             ; i++
        CPX  #4         ; i == 4 ?
        BNE  LOOP
        STAA $74        ; store the sum
        .END
```

Final values: $74 = 20 ($14)

Sample trace

Iteration	X	Read address	Element	A (sum after ADDA)
1	0	$70	6	6
2	1	$71	3	9
3	2	$72	9	18
4	3	$73	2	20

The sum is stored only once, after the loop ends.